

Database: A database is an organized collection of data stored in a computer system and usually controlled by a database management system (DBMS). The data in common databases is modeled in tables, making querying and processing efficient. Structured query language (SQL) is commonly used for data querying and writing.

The Database is an essential part of our life. We encounter several activities that involve our interaction with databases, for example in the bank, in the railway station, in school, in a grocery store, etc. These are the instances where we need to store a large amount of data in one place and fetch these data easily.

A database is a collection of data that is organized, which is also called structured data. It can be accessed or stored in a computer system. It can be managed through a Database Management System (DBMS), a software used to manage data. Database refers to related data in a structured form.

Application: Company Information, Account information, manufacturing, banking, finance transactions, telecommunications.

In a database, data is organized into tables consisting of rows and columns and it is indexed so data can be updated, expanded, and deleted easily. Computer databases typically contain file records data like transactions money in one bank account to another bank account, sales and customer details, fee details of students, and product details. There are different kinds of databases, ranging from the most prevalent approach, the relational database, to a distributed database, cloud database, and NoSQL databases.

- Relational Database:

A relational database is made up of a set of tables with data that fits into a predefined category.

- Distributed Database:

A distributed database is a database in which portions of the database are stored in multiple physical locations, and in which processing is dispersed or replicated among different points in a network.

- Cloud Database:

A cloud database is a database that typically runs on a cloud computing platform. Database service provides access to the database. Database services make the underlying software stack transparent to the user.

Characteristics of database

A number of characteristics distinguish the database approach from the much older approach of programming with files.

In traditional file processing, each user defines and implements the files needed for a specific software application as part of programming the application.

The main characteristics of the database approach versus the file-processing approach are the following:

- Self-describing nature of a database system.
- Insulation between programs and data, and data abstraction.
- Support of multiple views of the data.
- Sharing of data and multiuser transaction processing.

The database approach has some very characteristic features which are discussed in detail below:

Structured and Described Data:

- Fundamental feature of the database approach is that the database system does not only contain the data but also the complete definition and description of these data.
- These descriptions are basically details about the extent, the structure, the type and the format of all data and, additionally, the relationship between the data. This kind of stored data is called metadata ("data about data").

Separation of Data and Applications:

Application software does not need any knowledge about the physical data storage like encoding, format, storage place, etc. It only communicates with the management system of a database (DBMS) via a standardized interface with the help of a standardized language like SQL.

Data Integrity:

- Data integrity is a byword for the quality and the reliability of the data of a database system.
- In a broader sense data integrity includes also the protection of the database from unauthorized access (confidentiality) and unauthorized changes. Data reflect facts of the real world.

Transactions:

A transaction is a bundle of actions which are done within a database to bring it from one consistent state to a new consistent state. In between the data are inevitable inconsistent.

Data Persistence:

- Data persistence means that in a DBMS all data is maintained as long as it is not deleted explicitly.
- The life span of data needs to be determined directly or indirectly by the user and must not be dependent on system features.
- Additionally, data once stored in a database must not be lost. Changes of a database which are done by a transaction are persistent.
- When a transaction is finished even a system crash cannot put the data in danger.

Advantages and Disadvantages of DBMS:

Using a DBMS to manage data has many advantages:

Providing Backup and Recovery:

- A DBMS must provide facilities for recovering from hardware or software failures.
- The backup and recovery subsystem of the DBMS is responsible for recovery. For example, if the computer system fails in the middle of a complex update transaction, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the transaction started executing.

Data Independence:

- Application programs should be as independent as possible from details of data representation and storage. The DBMS can provide an abstract view of the data to insulate application code from such details.

Flexibility:

- It may be necessary to change the structure of a database as requirements change. For example, a new user group may emerge that needs information not currently in the database.

Efficient Data Access:

- A DBMS utilizes a variety of sophisticated techniques to store and retrieve data efficiently. This feature is especially important if the data is stored on external storage devices.

Data Integrity and Security:

- If data is always accessed through the DBMS, the DBMS can enforce integrity constraints on the data. For example, before inserting salary information for an employee, the DBMS

can check that the department budget is not exceeded. Also, the DBMS can enforce access controls that govern what data is visible to different classes of users.

Concurrent Access and Crash Recovery:

- A database system allows several users to access the database concurrently. Answering different questions from different users with the same (base) data is a central aspect of an information system. Such concurrent use of data increases the economy of a system.

Data Administration:

- When several users share the data, centralizing the administration of data can offer significant improvements. Experienced professionals, who understand the nature of the data being managed, and how different groups of users use it, can be responsible for organizing the data representation to minimize redundancy and fine-tuning the storage of the data to make retrieval efficient.

Reduced Application Development Time:

- DBMS supports many important functions that are common to many applications accessing data stored in the DBMS.
- This, in conjunction with the high-level interface to the data, facilitates quick development of applications.

Providing Multiple User Interfaces:

- Because many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces.
- These include query languages for casual users, programming language interfaces for application programmers, forms and command codes for parametric users, and menu-driven interfaces and natural language interfaces for standalone users.

The disadvantages of the database approach are summarized as follows:

- ***Danger of Overkill:*** For small and simple applications for single users a database system is often not advisable.
- ***Size:*** The complexity and breadth of functionality makes the DBMS an extremely large piece of software, occupying many megabytes of disk space and requiring substantial amounts of memory to run efficiently.

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit I: Database Management System

- **Complexity:** A database system creates additional complexity and requirements. The supply and operation of a database management system with several users and databases is quite costly and demanding.
- **Qualified Personnel:** The professional operation of a database system requires appropriately trained staff. Without a qualified database administrator nothing will work for long
- **Costs:** Through the use of a database system new costs are generated for the system itself but also for additional hardware and the more complex handling of the system.
- **Performance:** Typically, a File Based system is written for a specific application, such as invoicing. As result, performance is generally very good. However, the DBMS is written to be more general, to cater for many applications rather than just one. The effect is that some applications may not run as fast as they used to.
- **Higher impact of a failure:** The centralization of resources increases the vulnerability of the system. Since all users and applications rely on the availability of the DBMS, the failure of any component can bring operations to a halt.
- **Lower Efficiency:** A database system is a multi-use software which is often less efficient than specialized software which is produced and optimized exactly for one problem.

Difference between File System and DBMS

File system	DBMS
File system is a collection of data. Any management with the file system, user has to write the procedures.	DBMS is a collection of data and user is not required to write the procedures for managing the database.
File system gives the details of the data representation and Storage of data.	DBMS provides an abstract view of data that hides the details.
In File system storing and retrieving of data cannot be done efficiently.	DBMS is efficient to use since there are wide varieties of sophisticated techniques to store and retrieve the data.
Concurrent access to the data in the file system has many problems like Reading the file while other form of locking. deleting some information, updating some information	DBMS takes care of Concurrent access using some form of locking.
File system doesn't provide crash recovery mechanism. eg. While we are entering some data into the file if System crashes then content of the file is lost	DBMS has crash recovery mechanism, DBMS protects user from the effects of system failures.
Protecting a file under file system is very difficult.	DBMS has a good protection mechanism.

File Systems:

The traditionally used “File Systems” were nothing but a manual way of storing data as “Files”.

Characteristics of Traditional File Systems

Following are some of the characteristics of Traditional File Systems:

- The data of certain companies or organizations were kept as “Files”.
- The files stored in different departments were independent of each other, which caused severe data redundancy.
- Those files were developed using programming languages like COBOL, C, and C++.
- Each file includes information for a particular department or region, such as the library, tuition, and students’ exams.
- The traditional file system is way less flexible than DBMS and has many disadvantages.
- The maintenance of those files was also of high cost.
- Each of the units of “Files” used to be known as “Flat Files”.

Disadvantages of the Traditional File Systems

The following issues are related to the file-based approach:

- ***Distinguished and Isolated Data:*** Imagine a user needs information that is not possible to be provided using a single file, multiple different files were required, which are situated in different departments. So all the employees first need to manually and carefully check each of the files in each department and find the relationship between them to decipher the information that the user wants.
- ***Data Duplication / Data Redundancy*** – Due to the manual storing of data, the same data used to be present in multiple locations using the space in each of the Hard Disk, below are some following reasons for which Data Redundancy can be a major issue
 - Storing the same data multiple times not only wastes resources in every machine but also is costly to maintain and wastes time.
 - Loss of data integrity is another major issue of Data Redundancy, imagine someone’s address is present in multiple systems and he has applied to update the address, in one system the address gets updated but in the rest of them it remains the same, so the if someone from any different department where the data is not updated tries to send them any letter or something then it would go to a wrong address.
- ***Dependence on Data*** – Files and information were stored in a certain specific format in files which is hard coded by programmers in languages like C/C++, COBOL, etc. So if any of the file’s format changes then the programmers need to update the code every time and the

format of every piece of data stored in that file will be changed, which is a rigorous task for programmers.

- ***Data representation is challenging from the user's perspective*** – The data stored in files must be represented in such a manner that every user can understand that data clearly and easily, often it would become hectic as the programmers need to go through different systems and find the proper connection between data and merge them together and represent them in a more human-readable format
- ***Different file types*** – The file structure would vary based on the programming language that was used to store them, for example, if a system used COBOL to store certain data then the structure of that data would differ from the one written in C/C++.
- ***Data Protection*** – Data protection was very less due to different reasons like Data Redundancy, manual storing of data, easy access of confidential data by unauthorized parties, etc.
- ***Issues with Transactions*** – It didn't follow the ACID (Atomicity, Consistency, Isolation, and Durability) properties, for that if in the middle of any transaction the system crashed then it would leave the system in an inconsistent state.
- ***Concurrent issues*** – Two or more users can view the same file simultaneously, but the problem arises when they try to update the same file simultaneously.

Data model:

Data models describe how a database's logical structure is represented. In a database management system, data models are essential for introducing abstraction. Data models specify how data is linked to one another, as well as how it is handled and stored within the system.

A Database model defines the logical design and structure of a database. It defines how data will be stored, accessed, and updated in a database management system.

- As per your application's requirement, you can use a database model to define your database.
- The database model sets the rule, relationships, constraints, etc. to define how data is stored in the database.
- It's like creating a blueprint of your Database.
- There are different types of Database models and each one has its own set of features.
- You can define how you want to structure the application data using a database model.

Type of Database models

There are several different Database model types, some of them are old, while some of them are new, to cater to the new age requirements. Here is a list of the 7 popular Database models:

1. Hierarchical Model
2. Network Model
3. Entity-relationship Model
4. Relational Model
5. Object-oriented Model

1. Hierarchical Model

The hierarchical database model organizes data into a tree-like structure, with a single root, to which all the other data is linked.

- The hierarchy starts from the Root data, and expands like a tree, adding child nodes to the parent nodes.
- In this model, a child node will only have a single parent node.
- This model efficiently describes many real-world relationships like the index of a book, etc.
- IBM's Information Management System (IMS) is based on this model.

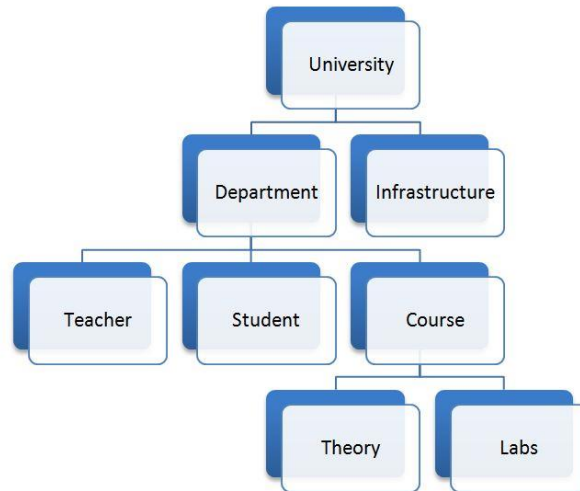
RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit I: Database Management System

- Data is organized into a tree-like structure with a one-to-many relationship between two different types of data, for example, one department can have many courses, many teachers, and of course many students



Advantages/Disadvantages of the Hierarchical Model

Here are a few points to mark the advantages and disadvantages of the Hierarchical database model:

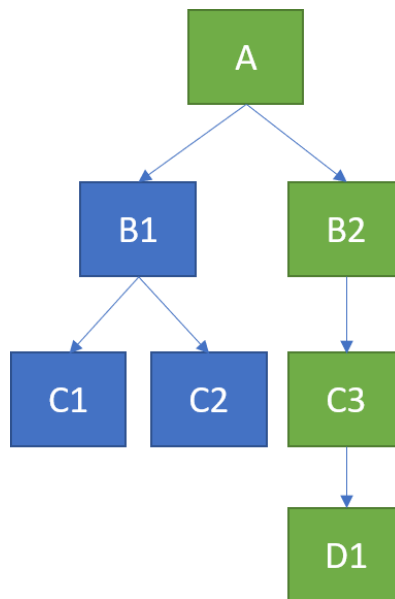
- Because it has one-to-many relationships between different types of data so it is easier and fast to fetch the data.
- But the Hierarchical model is less flexible.
- And it doesn't support many-to-many relationships.

2. Network Model

The Network Model is an extension of the Hierarchical model.

- In this model, data is organized more like a graph, and allowed to have more than one parent node.
- In the network database model, data is more related as more relationships are established in this database model.
- Also, as the data is more related, hence accessing the data is also easier and fast.
- This database model uses many-to-many data relationships.
- Integrated Data Store (IDS) is based on this database model.
- This was the most widely used database model before Relational Model was introduced.

- The implementation of the Network model is complex, and it's very difficult to maintain it.
- The Network model is difficult to modify also.
- You may want to explore this if you are developing some social networking applications, although the Graph Database model is new and is far better than the Network Database model.



Advantages of the Network Model

- It supports complex relationships
- It allows more flexibility

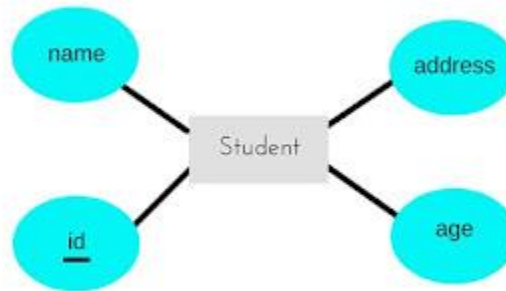
3. Entity-relationship Model

In this database model, relationships are created by dividing objects of interest into entities and their characteristics into attributes.

- Different entities are related using relationships.
- ER Models are defined to represent the relationships in pictorial form to make it easier for different stakeholders to understand.
- This model is good to design a database, which can then be turned into tables in a relational model.
- Let's take an example, if we have to design a School Database, then the Student will be an entity with attributes name, age, address, etc. As an Address is generally complex, it can be

another entity with attributes street, pincode, city, etc, and there will be a relationship between them.

- Relationships can also be of different types. You can learn about ER Diagrams in detail if you want to learn about entities and relationships.



Advantages of the ER Model

- It is easy to understand and design.
- Using the ER model we can represent data structures easily.
- As the ER model cannot be directly implemented into a database model, it is just a step toward designing the relational database model.

4. Relational Model

In this model, data is organized in two-dimensional tables and the relationship is maintained by storing a common field.

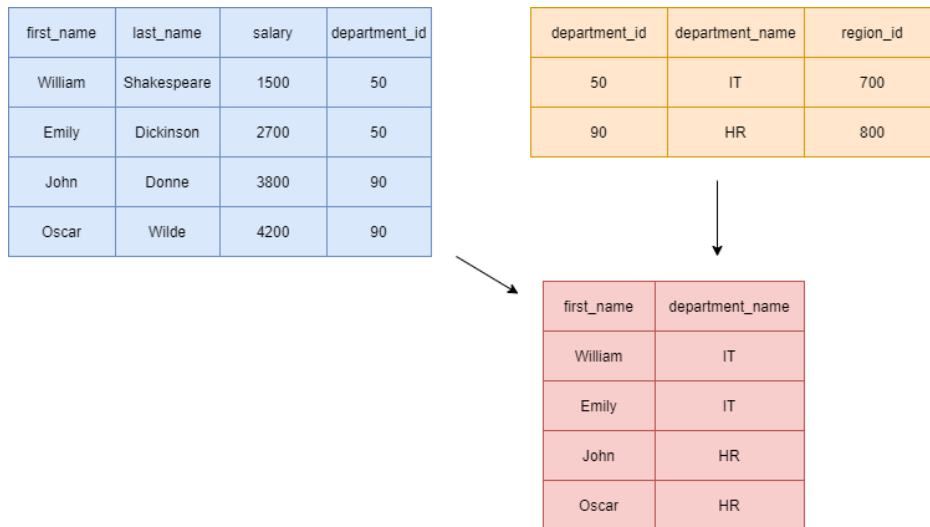
- This model was introduced by E.F Codd in 1970, and since then it has been the most widely used database model.
- The basic structure of data in the relational model is tables. All the information related to a particular type is stored in rows of that table.
- Hence, tables are also known as relations in the relational model.
- You can design tables, normalize them to reduce data redundancy, and use Structured Query language or SQL to access data from the tables.
- Some of the most popular databases are based on this database model. For example, Oracle, MySQL, etc.

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit I: Database Management System



Advantages of the Relational Model

- It's simple and easy to implement.
- Popular database software is available for this database model.
- It supports SQL using which you can easily query the data.

5. Object-oriented Model

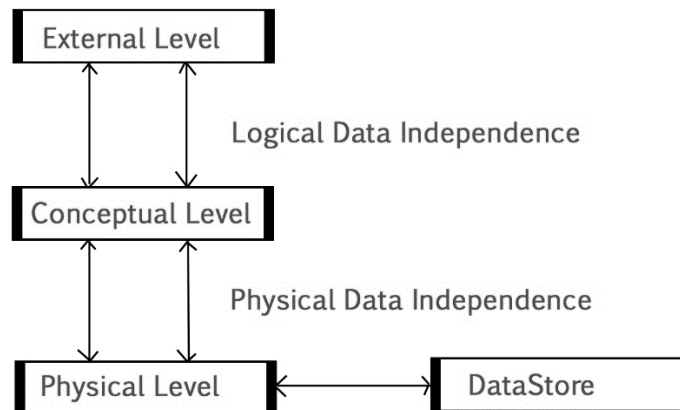
- In this model, data is stored in the form of objects.
- The behavior of the object-oriented database model is just like object-oriented programming.
- A very popular example of an Object Database management system or ODBMS is MongoDB which is also a NoSQL database.
- This database model is not mature enough as compared to the relational database model.

Advantages of the Object-oriented Model

- It can easily support complex data structures, with relationships.
- It also supports features like Inheritance, Encapsulation, etc.

3-Tier Architecture in DBMS

DBMS 3-tier architecture divides the complete system into three inter-related but independent modules.



1. **Physical Level:** At the physical level, the information about the location of database objects in the data store is kept. Various users of DBMS are unaware of the locations of these objects. In simple terms, physical level of a database describes how the data is being stored in secondary storage devices like disks and tapes and also gives insights on additional storage details.
2. **Conceptual Level:** At conceptual level, data is represented in the form of various database tables. For Example, STUDENT database may contain STUDENT and COURSE tables which will be visible to users but users are unaware of their storage. Also referred as logical schema, it describes what kind of data is to be stored in the database.
3. **External Level:** An external level specifies a view of the data in terms of conceptual level tables. Each external level view is used to cater to the needs of a particular category of users. For Example, FACULTY of a university is interested in looking course details of students, STUDENTS are interested in looking at all details related to academics, accounts, courses and hostel details as well. So, different views can be generated for different users. The main focus of external level is data abstraction.

Data independence means a change of data at one level should not affect another level. Two types of data independence are present in this architecture:

Physical Data Independence: Any change in the physical location of tables and indexes should not affect the conceptual level or external view of data. This data independence is easy to achieve and implemented by most of the DBMS.

Conceptual Data Independence: The data at conceptual level schema and external level schema must be independent. This means a change in conceptual schema should not affect external schema. e.g.; Adding or deleting attributes of a table should not affect the user's view of the table. But this type of independence is difficult to achieve as compared to physical data independence because the changes in conceptual schema are reflected in the user's view.

Reference Questions:

Q1. What is DBMS? What are the characteristics of it?

Q2. Compare File System with DBMS.

Q3. What do you mean by schema and instance of database?

Q4. What is Data Model? Explain its classifications.

Q5. Explain 3-tier database architecture?

ER Model:

The Entity Relational Model is a model for identifying entities to be represented in the database and representation of how those entities are related. The ER data model specifies enterprise schema that represents the overall logical structure of a database graphically.

The Entity Relationship Diagram explains the relationship among the entities present in the database. ER models are used to model real-world objects like a person, a car, or a company and the relation between these real-world objects. In short, ER Diagram is the structural format of the database.

Why Use ER Diagrams In DBMS?

- ER diagrams are used to represent the E-R model in a database, which makes them easy to be converted into relations (tables).
- ER diagrams provide the purpose of real-world modeling of objects which makes them intently useful.
- ER diagrams require no technical knowledge and no hardware support.
- These diagrams are very easy to understand and easy to create even for a naive user.
- It gives a standard solution for visualizing the data logically.

Symbols Used in ER Model

- ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:
- Rectangles: Rectangles represent Entities in ER Model.
- Ellipses: Ellipses represent Attributes in ER Model.
- Diamond: Diamonds represent Relationships among Entities.
- Lines: Lines represent attributes to entities and entity sets with other relationship types.
- Double Ellipse: Double Ellipses represent Multi-Valued Attributes.
- Double Rectangle: Double Rectangle represents a Weak Entity.

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit II: Database Management System

Symbol Name	Symbol	Represents
Rectangles		Represents Entity
Ellipses		Represents Attribute
Diamonds		Represents Relationship
Lines		Links Attribute(s) to entity set(s) or Entity set(s) to Relationship set(s)
Double Ellipses		Represents Multivalued Attributes
Primary key		Represents Key Attributes / Single Valued Attributes

Strong Entity:

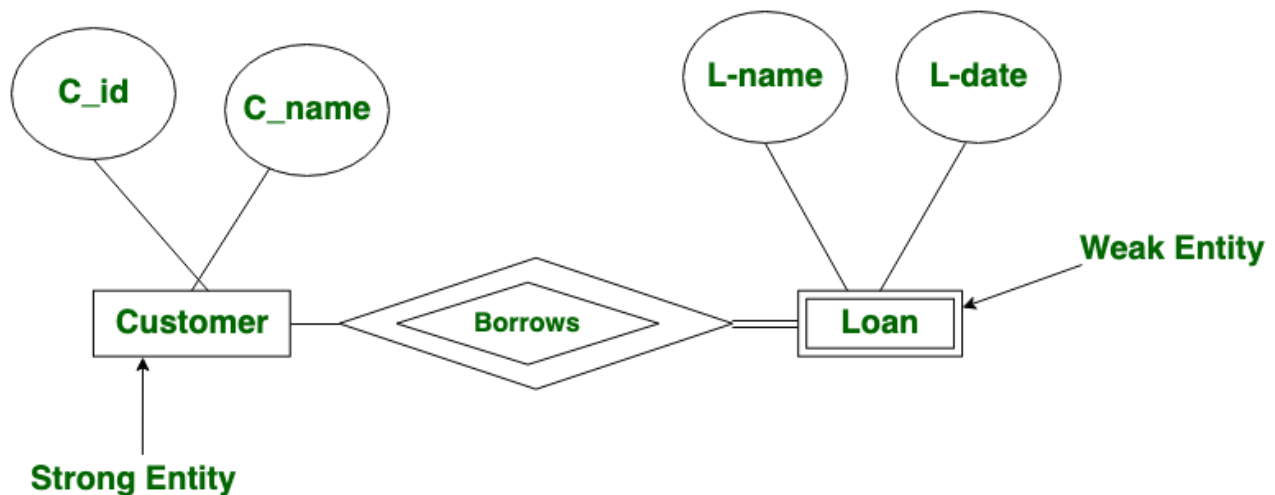
A strong entity is not dependent on any other entity in the schema. A strong entity will always have a primary key. Strong entities are represented by a single rectangle. The relationship of two strong entities is represented by a single diamond.

Various strong entities, when combined together, create a strong entity set.

Weak Entity:

A weak entity is dependent on a strong entity to ensure its existence. Unlike a strong entity, a weak entity does not have any primary key. It instead has a partial discriminator key. A weak entity is represented by a double rectangle.

The relation between one strong and one weak entity is represented by a double diamond. This relationship is also known as identifying relationship.



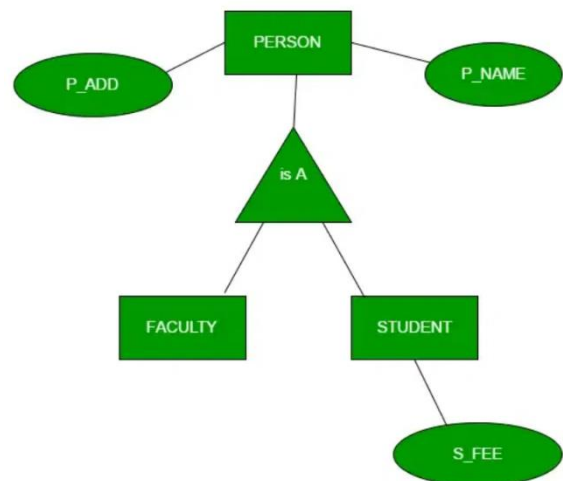
Generalization , Specialization and Aggregation

Using the ER model for bigger data creates a lot of complexity while designing a database model, So in order to minimize the complexity Generalization, Specialization, and Aggregation were introduced in the ER model and these were used for data abstraction in which an abstraction mechanism is used to hide details of a set of objects. Some of the terms were added to the Enhanced ER Model, where some new concepts were added. These new concepts are:

- Generalization
- Specialization
- Aggregation

Generalization

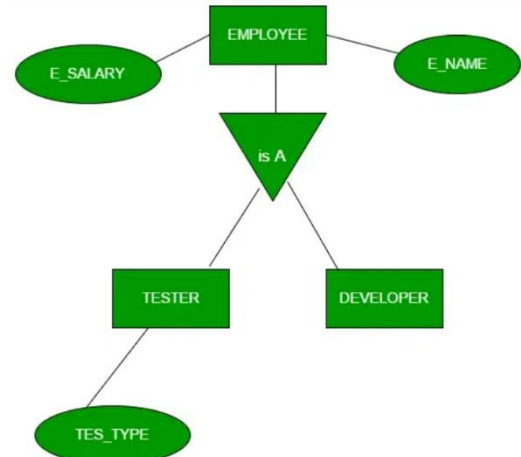
Generalization is the process of extracting common properties from a set of entities and creating a generalized entity from it. It is a bottom-up approach in which two or more entities can be generalized to a higher-level entity if they have some attributes in common. For Example, STUDENT and FACULTY can be generalized to a higher-level entity called PERSON as shown in Figure 1. In this case, common attributes like P_NAME, and P_ADD become part of a higher entity (PERSON), and specialized attributes like S_FEE become part of a specialized entity (STUDENT).



Generalization is also called as ‘ Bottom-up approach’.

Specialization

In specialization, an entity is divided into sub-entities based on its characteristics. It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities. For Example, an EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER, etc. as shown in Figure 2. In this case, common attributes like E_NAME, E_SAL, etc. become part of a higher entity (EMPLOYEE), and specialized attributes like TES_TYPE become part of a specialized entity (TESTER).

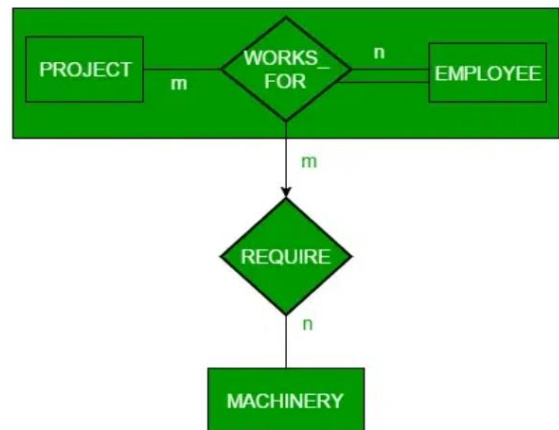


Specialization is also called as ” Top-Down approach”.

Aggregation

An ER diagram is not capable of representing the relationship between an entity and a relationship which may be required in some scenarios. In those cases, a relationship with its corresponding entities is aggregated into a higher-level entity. Aggregation is an abstraction through which we can represent relationships as higher-level entity sets.

For Example, an Employee working on a project may require some machinery. So, REQUIRE relationship is needed between the relationship WORKS_FOR and entity MACHINERY. Using aggregation, WORKS_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into a single entity and relationship REQUIRE is created between the aggregated entity and MACHINERY.



Integrity Constraints:

Integrity constraints are a set of rules. It is used to maintain the quality of information. Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected. Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint -

- Domain Constraints
- Key Constraints
- Entity Integrity Constraints
- Referential Integrity Constraints
- Tuple Uniqueness Constraints

Domain Constraints

In a database table, domain constraints are guidelines that specify the acceptable values for a certain property or field. These restrictions guarantee data consistency and aid in preventing the entry of inaccurate or inconsistent data into the database. The following are some instances of domain restrictions in a DBMS –

1. Data type constraints – These limitations define the kinds of data that can be kept in a column. A column created as VARCHAR can take string values, but a column specified as INTEGER can only accept integer values.
2. Length Constraints – These limitations define the largest amount of data that may be put in a column. For instance, a column with the definition VARCHAR(10) may only take strings that are up to 10 characters long.
3. Range constraints – The allowed range of values for a column is specified by range restrictions. A column designated as DECIMAL(5,2), for example, may only take decimal values up to 5 digits long, including 2 decimal places.
4. Nullability constraints – Constraints on a column's capacity to accept NULL values are known as nullability constraints. For instance, a column that has the NOT NULL definition cannot take NULL values.
5. Unique constraints – Constraints that require the presence of unique values in a column or group of columns are known as unique constraints. For instance, duplicate values are not allowed in a column with the UNIQUE definition.

6. Check constraints – Constraints for checking data: These constraints outline a requirement that must hold for any data placed into the column. For instance, a column with the definition CHECK (age > 0) can only accept ages that are greater than zero.
7. Default constraints – Constraints by default: Default constraints automatically assign a value to a column in case no value is provided. For example, a column with a DEFAULT value of 0 will have 0 as its value if no other value is specified.

Key Constraints

Key constraints are regulations that a DBMS uses to ensure data accuracy and consistency in a database. They define how the values in a table's one or more columns are related to the values in other tables, making sure that the data remains correct.

In DBMS, there are several key constraint kinds, including –

1. Primary Key Constraint – A primary key constraint is an individual identifier for each record in a database. It guarantees that each database entry contains a single, distinct value—or a pair of values—that cannot be null—as its method of identification.
2. Foreign Key Constraint – Reference to the primary key in another table is a foreign key constraint. It ensures that the values of a column or set of columns in one table correspond to the primary key column(s) in another table.
3. Unique Constraint – In a database, a unique constraint ensures that no two values inside a column or collection of columns are the same.

Entity Integrity Constraints

A database management system uses entity integrity constraints (EICs) to enforce rules that guarantee a table's primary key is unique and not null. The consistency and integrity of the data in a database are maintained by EICs, which are created to stop the formation of duplicate or incomplete entries.

Each item in a table in a relational database is uniquely identified by one or more fields known as the primary key. EICs make a guarantee that every row's primary key value is distinct and not null. Take the "Employees" table, for instance, which has the columns "EmployeeID" and "Name." The table's primary key is the EmployeeID column. An EIC on this table would make sure that each row's unique EmployeeID value is there and that it is not null.

If you try to insert an entry with a duplicate or null EmployeeID, the database management system will reject the insertion and produce an error. This guarantees that the information in the table is correct and consistent.

EICs are a crucial component of database architecture and assist guarantee the accuracy and dependability of the data contained in a database.

Referential Integrity Constraints

A database management system will apply referential integrity constraints (RICs) in order to preserve the consistency and integrity of connections between tables. By preventing links between entries that don't exist from being created or by removing records that have related records in other tables, RICs guarantee that the data in a database is always consistent.

By the use of foreign keys, linkages between tables are created in relational databases. A column or collection of columns in one table that is used as a foreign key to access the primary key of another table. RICs make sure there are no referential errors and that these relationships are legitimate.

Consider the "Orders" and "Customers" tables as an illustration. The primary key column in the "Customers" database corresponds to the foreign key field "CustomerID" in the "Orders" dataset. A RIC on this connection requires that each value in the "CustomerID" column of the "Orders" database exist in the "Customers" table's primary key column.

If an attempt was made to insert a record into the "Orders" table with a non-existent "CustomerID" value, the database management system would reject the insertion and notify the user of an error.

Similar to this, the database management system would either prohibit the deletion or cascade the deletion in order to ensure referential integrity if a record in the "Customers" table was removed and linked entries in the "Orders" table.

In general, RICs are a crucial component of database architecture and assist guarantee that the information contained in a database is correct and consistent throughout time.

NOT NULL Constraint

By default, a column can hold NULL values. If you do not want a column to have a NULL value, use the NOT NULL constraint. It restricts a column from having a NULL value.

```
CREATE TABLE Student
(
    s_id int NOT NULL,
    name varchar(60),
    age int
);
```

UNIQUE Constraint

It ensures that a column will only have unique values. A UNIQUE constraint field cannot have any duplicate data. It prevents two records from having identical values in a column.

```
CREATE TABLE Student
(
    s_id int NOT NULL,
    name varchar(60),
    age int NOT NULL UNIQUE
);
```

CHECK Constraint

CHECK constraint is used to restrict the value of a column between a range. It performs check on the values, before storing them into the database. Its like condition checking before saving data into a column.

```
CREATE table Student(
    s_id int NOT NULL CHECK(s_id > 0),
    Name varchar(60) NOT NULL,
    Age int
);
```

KEYS:

A key is an attribute of a table which helps to identify a row.

There can be many different types of keys:

Super Key or Candidate Key: Super Key is defined as a set of attributes within a table that uniquely identifies each record within a table. Super Key is a superset of Candidate key.

Example: {Employee_Id}, {Employee_Id, Employee_Name}, {Employee_Id, Employee_Name, Address} etc.

Candidate keys are defined as the set of fields from which primary key can be selected. It is an attribute or set of attribute that can act as a primary key for a table to uniquely identify each record in that table.

Example: Employee_Id, License_Number and Passport_Number are candidate keys

Primary Key: Primary key is a candidate key of the table selected to identify each record uniquely in table. Primary key does not allow null value in the column and keeps unique values throughout the column. In above

Example: Employee_Id is a primary key of Employee table.

Unique Key: Unique key is similar to primary key and does not allow duplicate values in the column. It has below differences in comparison of primary key:

1. It allows one null value in the column.
2. By default, it creates a non clustered index on heap tables.

Alternate Key: Out of all candidate keys, only one gets selected as primary key, remaining keys are known as alternate or secondary keys.

Example: License_Number and Passport_Number are alternate keys.

Composite Key: Composite key (also known as compound key or concatenated key) is a group of two or more columns that identifies each row of a table uniquely. Individual column of composite key might not be able to uniquely identify the record. It can be a primary key or candidate key also.

Example: In salary table, Employee_Id and Salary_Month_Year are combined together to identify each row uniquely in Salary table. Independently Employee_Id or Salary_Month_Year column

cannot identify each row uniquely. We can create a composite primary key on Salary table using Employee_Id and Salary_Month_Year columns.

Foreign Key: A foreign key is a key used to link two tables together. This is sometimes also called as a referencing key. A Foreign Key is a column or a combination of columns whose values match a Primary Key in a different table.

In a relationship between two tables, a primary key of one table is referred as a foreign key in another table. Foreign key can have duplicate values in it and can also keep null values if column is defined to accept nulls.

Example: Employee_Id (primary key of Employee table) is a foreign key in Salary table.

Reference Questions:

- Q1. Explain Entity Relationship Model? Explain using ER diagram.
- Q2. What do you mean by Relationship between database? Explain its types
- Q3. What do you mean by Specialization, Generalization, and Aggregation?
- Q4. What is Data Integrity Constraints?
- Q5. Explain Relational Database Design?

Functional Dependency

Definition: A functional dependency occurs when one attribute in a relation uniquely determines another attribute. This can be written $A \rightarrow B$ which would be the same as stating “B is functionally dependent upon A”.

OR

A functional dependency is a set of rule between set of attributes. If R is a relation with attributes X and Y, a functional dependency between the attributes is represented as $X \rightarrow Y$, which specifies Y is functionally dependent on X. Here X is a determinant set and Y is a dependent attribute. Each value of X is associated precisely with one Y value.

Example:-

RollNO $\rightarrow$ Name, Age, DOB, Email

SSN $\rightarrow$ Name, Email, Address

ISBN $\rightarrow$ Aut_Name, Price, PublishersName

On all above example RollNo, SSN, ISBN are the primary key of relation and here non key attribute (Name, age..) are functionally dependent on primary key.

RollNo, SSN, ISBN are called determinant.

NORMALIZATION:

Normalization was first proposed by Codd as an integral part of the relational model. It encompasses a set of procedures designed to eliminate non simple domains (non-atomic values) and the redundancy (duplication) of data, which in turn prevents data manipulation anomalies and loss of data integrity. The most common forms of normalization applied to databases are called the normal forms.

While designing a database out of an entity–relationship model, the main problem existing in that “raw” database is redundancy. Redundancy is storing the same data item in more one place. A redundancy creates several problems like the following:

1. Extra storage space: storing the same data in many places takes large amount of disk space.
2. Entering same data more than once during data insertion.
3. Deleting data from more than one place during deletion.
4. Modifying data in more than one place.
5. Anomalies may occur in the database if insertion, deletion, modification etc are no done properly. It creates inconsistency and unreliability in the database.

To solve this problem, the “raw” database needs to be normalized. This is a step by step process of removing different kinds of redundancy and anomaly at each step. At each step a specific rule is followed to remove specific kind of impurity in order to give the database a slim and clean look.

Un-Normalized Form (UNF)

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

If a table contains non-atomic values at each row, it is said to be in UNF. An atomic value is something that can not be further decomposed. A non-atomic value, as the name suggests, can be further decomposed and simplified. Consider the following table:

Emp-Id	Emp-Name	Month	Sales	Bank-Id	Bank-Name
E01	AA	Jan	1000	B01	SBI
		Feb	1200		
		Mar	850		
E02	BB	Jan	2200	B02	UTI
		Feb	2500		
E03	CC	Jan	1700	B01	SBI
		Feb	1800		
		Mar	1850		
		Apr	1725		

In the sample table above, there are multiple occurrences of rows under each key Emp-Id. Although considered to be the primary key, Emp-Id cannot give us the unique identification facility for any single row. Further, each primary key points to a variable length record (3 for E01, 2 for E02 and 4 for E03).

First Normal Form (1NF):

A relation is said to be in 1NF if it contains no non-atomic values and each row can provide a unique combination of values. The above table in UNF can be processed to create the following table in 1NF.

Emp-Id	Emp-Name	Month	Sales	Bank-Id	Bank-Name
E01	AA	Jan	1000	B01	SBI
E01	AA	Feb	1200	B01	SBI
E01	AA	Mar	850	B01	SBI
E02	BB	Jan	2200	B02	UTI
E02	BB	Feb	2500	B02	UTI
E03	CC	Jan	1700	B01	SBI
E03	CC	Feb	1800	B01	SBI
E03	CC	Mar	1850	B01	SBI
E03	CC	Apr	1725	B01	SBI

As you can see now, each row contains unique combination of values. Unlike in UNF, this relation contains only atomic values, i.e. the rows can not be further decomposed, so the relation is now in 1NF.

Second Normal Form (2NF):

A relation is said to be in 2NF if it is already in 1NF and each and every attribute fully depends on the primary key of the relation. Speaking inversely, if a table has some attributes which is not dependant on the primary key of that table, then it is not in 2NF.

Let us explain. Emp-Id is the primary key of the above relation. Emp-Name, Month, Sales and Bank-Id all depend upon Emp-Id. But the attribute Bank-Name depends on Bank-Id, which is

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

not the primary key of the table. So the table is in 1NF, but not in 2NF. If this position can be removed into another related relation, it would come to 2NF.

Emp-Id	Emp-Name	Month	Sales	Bank-Id
E01	AA	JAN	1000	B01
E01	AA	FEB	1200	B01
E01	AA	MAR	850	B01
E02	BB	JAN	2200	B02
E02	BB	FEB	2500	B02
E03	CC	JAN	1700	B01
E03	CC	FEB	1800	B01
E03	CC	MAR	1850	B01
E03	CC	APR	1726	B01

Bank-Id	Bank-Name
B01	SBI
B02	UTI

After removing the portion into another relation we store lesser amount of data in two relations without any loss information. There is also a significant reduction in redundancy.

Third Normal Form (3NF):

A relation is said to be in 3NF, if it is already in 2NF and there exists no transitive dependency in that relation. Speaking inversely, if a table contains transitive dependency, then it is not in 3NF, and the table must be split to bring it into 3NF.

What is a transitive dependency? Within a relation if we see

$A \rightarrow B$ [B depends on A]

And

$B \rightarrow C$ [C depends on B]

Then we may derive

$A \rightarrow C$ [C depends on A]

Such derived dependencies hold well in most of the situations. For example if we have

$\text{Roll} \rightarrow \text{Marks}$

And

$\text{Marks} \rightarrow \text{Grade}$

Then we may safely derive

$\text{Roll} \rightarrow \text{Grade}$.

This third dependency was not originally specified but we have derived it.

The derived dependency is called a transitive dependency when such dependency becomes improbable. For example we have been given

$\text{Roll} \rightarrow \text{City}$

And

$\text{City} \rightarrow \text{STD Code}$

If we try to derive $\text{Roll} \rightarrow \text{STD Code}$ it becomes a transitive dependency, because obviously the STDCode of a city cannot depend on the roll number issued by a school or college. In such a case

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

the relation should be broken into two, each containing one of these two dependencies:

Roll $\rightarrow$ City

And

City $\rightarrow$ STD code

Boyce-Code Normal Form (BCNF):

A relationship is said to be in BCNF if it is already in 3NF and the left hand side of every dependency is a candidate key. A relation which is in 3NF is almost always in BCNF. These could be same situation when a 3NF relation may not be in BCNF the following conditions are found true.

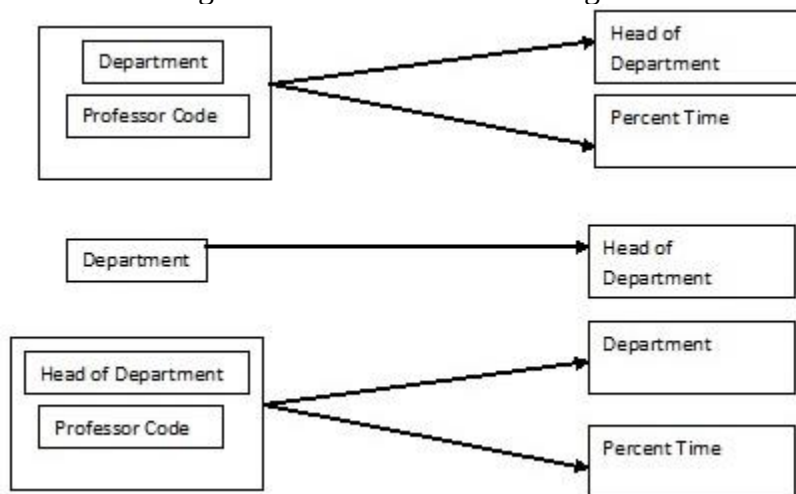
1. The candidate keys are composite.
2. There are more than one candidate keys in the relation.
3. There are some common attributes in the relation.

Professor Code	Department	Head of Dept.	Percent Time
P1	Physics	Ghosh	50
P1	Mathematics	Krishnan	50
P2	Chemistry	Rao	25
P2	Physics	Ghosh	75
P3	Mathematics	Krishnan	100

Consider, as an example, the above relation. It is assumed that:

1. A professor can work in more than one department
2. The percentage of the time he spends in each department is given.
3. Each department has only one Head of Department.

The relation diagram for the above relation is given as the following:



The given relation is in 3NF. Observe, however, that the names of Dept. and Head of Dept. are duplicated. Further, if Professor P2 resigns, rows 3 and 4 are deleted. We lose the information that Rao is the Head of Department of Chemistry.

The normalization of the relation is done by creating a new relation for Dept. and Head of Dept. and deleting Head of Dept. form the given relation. The normalized relations are shown in the following.

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

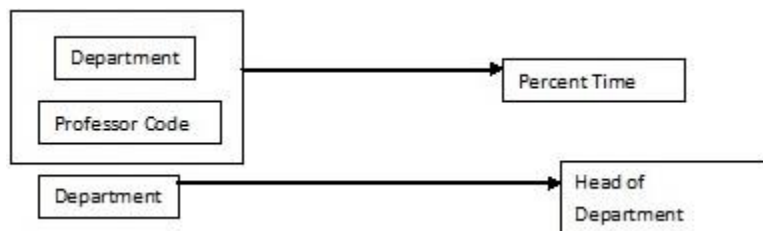
Subject:

Unit III: Database Management System

Professor Code	Department	Percent Time
P1	Physics	50
P1	Mathematics	50
P2	Chemistry	25
P2	Physics	75
P3	Mathematics	100

Department	Head of Dept.
Physics	Ghosh
Mathematics	Krishnan
Chemistry	Rao

See the dependency diagrams for these new relations.



Fourth Normal Form (4NF):

When attributes in a relation have multi-valued dependency, further Normalization to 4NF and 5NF are required. Let us first find out what multi-valued dependency is.

A multi-valued dependency is a typical kind of dependency in which each and every attribute within a relation depends upon the other, yet none of them is a unique primary key.

We will illustrate this with an example. Consider a vendor supplying many items to many projects in an organization. The following are the assumptions:

1. A vendor is capable of supplying many items.
2. A project uses many items.
3. A vendor supplies to many projects.
4. An item may be supplied by many vendors.

A multi valued dependency exists here because all the attributes depend upon the other and yet none of them is a primary key having unique value.

Vendor Code	Item Code	Project No.
V1	I1	P1
V1	I2	P1
V1	I1	P3
V1	I2	P3
V2	I2	P1
V2	I3	P1

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

V3	I1	P2
V3	I1	P3

The given relation has a number of problems. For example:

1. If vendor V1 has to supply to project P2, but the item is not yet decided, then a row with a blank for item code has to be introduced.
2. The information about item I1 is stored twice for vendor V3.

Observe that the relation given is in 3NF and also in BCNF. It still has the problem mentioned above. The problem is reduced by expressing this relation as two relations in the Fourth Normal Form (4NF). A relation is in 4NF if it has no more than one independent multi valued dependency or one independent multi valued dependency with a functional dependency.

The table can be expressed as the two 4NF relations given as following. The fact that vendors are capable of supplying certain items and that they are assigned to supply for some projects in independently specified in the 4NF relation.

Vendor-Supply

Vendor Code	Item Code
V1	I1
V1	I2
V2	I2
V2	I3
V3	I1

Vendor-Project

Vendor Code	Project No.
V1	P1
V1	P3
V2	P1
V3	P2

Fifth Normal Form (5NF):

These relations still have a problem. While defining the 4NF we mentioned that all the attributes depend upon each other. While creating the two tables in the 4NF, although we have preserved the dependencies between Vendor Code and Item code in the first table and Vendor Code and Item code in the second table, we have lost the relationship between Item Code and Project No. If there were a primary key then this loss of dependency would not have occurred. In order to revive this relationship we must add a new table like the following. Please note that during the entire process of normalization, this is the only step where a new table is created by joining two attributes, rather than splitting them into separate tables.

Project No.	Item Code
P1	I1
P1	I2
P2	I1

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

P3	11
P3	13

Let us finally summarize the normalization steps we have discussed so far.

Input Relation	Transformation	Output Relation
All Relations	Eliminate variable length record. Remove multi-attribute lines in table.	1NF
1NF Relation	Remove dependency of non-key attributes on part of a multi-attribute key.	2NF
2NF	Remove dependency of non-key attributes on other non-key attributes.	3NF
3NF	Remove dependency of an attribute of a multi attribute key on an attribute of another (overlapping) multi-attribute key.	BCNF
BCNF	Remove more than one independent multi-valued dependency from relation by splitting relation.	4NF
4NF	Add one relation relating attributes with multi-valued dependency.	5NF

STRUCTURED QUERY LANGUAGE (SQL):

SQL is a standard language for accessing databases. It is a high level language that supports definition, manipulation and control of data in relational databases. SQL is the standard language for communicating with relational database management systems. SQL statements are used to retrieve data from the database as well as perform tasks such as adding updating and deleting the data. Some common relational database management systems that use SQL are: Oracle, Sybase, MS SQL Server, MS Access, MySQL, etc.

Characteristics or advantage of SQL:

- 1 It provides greater degree of abstraction than procedural language.
- 2 It enables the user to deal with a number of database systems available.
- 3 Application written in SQL is portable that is can be used at any platform.
- 4 It is simple and easy learns but handles complex situation.
- 5 It is used to define data structure, control access to the data, delete, insert & modify occurrence of the data.

BASIC STRUCTURE:

The basic structure of an SQL expression consists of three clauses:

```
SELECT field1 ,field2,... FROM table
[WHERE "condition"]
[GROUP BY "field"]
[ORDER BY "field"]
```

Where:

The **select** statement is used to query the database and retrieve the fields that you specify. You can select as many fields (column names) as you want, or use the asterisk symbol "*" to select all fields.

The **from** statement specifies the table names that will be queried to retrieve the desired data.

The **where** clause (optional) specifies which data values or rows will be returned or displayed, based on the criteria you specify.

The **group by** clause (optional) organizes data into groups.

The **order by** clause (optional) sorts the data by the field specified.

DATA TYPE:

SQL data type is an attribute that specifies type of data of any object. Each column, variable and expression has related data type in SQL. The object can hold like integer data, character data, date and time data, binary strings, and so on. The data type is a label and a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.

No	Data Type	Description
1	numeric(p,s)	Where p is a precision value; s is a scale value. For example, numeric (6, 2) is a number that has 4 digits before the decimal and 2 digits after the decimal.
2	char(x)	Where x is the number of characters to store. This data type is space padded to fill the number of characters specified.

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

3	int	Integer value
4	varchar2(x)	Where x is the number of characters to store. This data type does NOT space pad.
5	decimal(p,s)	Where p is a precision value, s is a scale value.
6	date	Stores year, month, and day values.
7	time	Stores the hour, minute, and second values.
8	float(p)	Where p is a precision value.

OPERATOR:

Operators are used to specify conditions in an SQL statement and to serve as conjunctions for multiple conditions in a statement.

1. Arithmetic operator:

SQL arithmetic operators are: + (addition), - (subtraction), * (multiplication), / (division), % (modulus)

2. Comparison operator:

SQL comparison operators are: > (greater than), < (less than), <= (less than or equal to), >= (greater than or equal to), != (not equal to), = (equal to)

3. Logical operator:

a) Between operator:

The operator BETWEEN and AND are used to compare data for a range of values. Column value is between two values, including the end values specified in the range.

Syntax:

```
Select column_name(s) from table_name  
where column_name between value1 and value2 ;
```

Example:

To find the names of the employee whose salary between 10000 and 20000.

```
Sql> select * from emp  
where esal between 10000 and 20000 ;
```

Not between operator:

Syntax:

```
Select column_name(s) from table_name  
where column_name not between value1 and value2 ;
```

Example:

To find the names of the employee whose salary not between 10000 and 20000.

```
Sql> select * from emp  
where esal not between 10000 and 20000 ;
```

b) In operator:

The IN operator is used when you want to compare a column with more than one value. It is similar to an OR condition.

Syntax:

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

```
Select column_name(s) from table_name
where column_name in( value1 and value2, ...);
```

Example:

To find the names of the employee whose lives in indore, mhow and bhopal.

```
Sql> select * from emp
      where ecity in('indore', 'mhow', 'bhopal');
```

Not in operator:

Syntax:

```
Select column_name(s) from table_name
where column_name not in ( value1 and value2, ...);
```

Example:

To find the names of the employee whose do not live in indore, mhow and bhopal.

```
Sql> select * from emp
      where ecity not in ('indore', 'mhow', 'bhopal');
```

c) Like operator:

The like operator is used to list all rows in a table whose column values match a specified pattern. It is useful when you want to search rows to match a specific pattern, or when you do not know the entire value. For this purpose we use a wildcard character '%'. There is another wildcard character you can use with like operator. It is the underscore character '_'. In a search string, the underscore signifies a single character.

Syntax:

```
Select column_name from table_name
where column_name like '%';
```

Example:

i) To select the entire employee whose name begins with 'S'.

```
sql> select * from emp
      where ename like 's%'
```

ii) To select the entire employee whose name ends with 'S'.

```
sql> select * from emp
      where ename like '%s'
```

iii) To display all the names with 'a' second character.

```
sql> select * from emp
      where ename like '_a%'
```

iv) To select the entire employee whose name contains character 'A'.

```
sql> select * from emp
      where ename like '%a%'
```

DATA LANGUAGE:

SQL is a language of database, it includes database creation, deletion, fetching rows and modifying rows etc. The standard SQL commands to interact with relational databases are create, select, insert, update, delete and drop. These commands can be classified into groups based on their nature.

DATA DEFINITION LANGUAGE (DDL):

A DDL is a language used to define data structures and modify data. A Data Definition Language has a pre-defined syntax for describing data. By this command we can create, alter and drop the structure of table view and index. Basic DDL commands are:

a) CREATE Command:

By create command, we can create database.

Syntax:

```
create database database name ;
```

Example:

```
Sql> create database iiscs ;
```

Creating a basic table involves naming the table and defining its columns and each column's data type. The sql create table statement is used to create a new table.

Syntax:

```
create table table_name  
( column_name1 data_type(size) constraint,  
  column_name2 data_type(size) constraint,  
  column_name3 data_type(size) constraint,  
  ....  
);
```

Example:

```
Sql> create table emp  
( eno      number(12)    primary key ,  
  ename    varchar2(15)  NOT NULL ,  
  esal     number(18)    check(esal>10000) ,  
  ecity    varchar(18) ,  
);
```

b) ALTER Command:

The sql alter table command is used to add, delete or modify columns in an existing table. We can also use alter table command to add and drop various constraints on an existing table.

i. The basic syntax of alter table to add a new column in an existing table is as follows:

Syntax:

```
alter table table_name  
add (column_name datatype(size)) ;
```

Example:

```
Sql> alter table emp  
add( eage number(12) ) ;
```

ii. The basic syntax of alter table to change the data type of a column in a table is as follows:

Syntax:

```
alter table table_name  
modify (column_name New datatype(size)) ;
```

Example:

```
Sql> alter table emp  
add( ename varchar2(20) ) ;
```

iii. The basic syntax of alter table to drop column in an existing table is as follows:

Syntax:

```
alter table table_name  
drop column column_name ;
```

Example:

```
Sql> alter table emp  
drop column eage ;
```

iv. The basic syntax of alter table to add primary key constraint to a table is as follows:

Syntax:

```
alter table table_name  
add primary key (column_name) ;
```

Example:

```
Sql> alter table emp  
add primary key(eno) ;
```

v. The basic syntax of alter table to drop primary key constraint from a table is as follows:

Syntax:

```
alter table table_name  
drop primary key ;
```

Example:

```
Sql> alter table emp  
drop primary key ;
```

c) DROP Command:

Tables, index, view and databases can easily be deleted /removed with the drop statement.

i. The drop database statement is used to delete a database.

Syntax:

```
drop database DatabaseName ;
```

Example:

```
Sql> drop database iiscs ;
```

ii. The drop table statement is used to delete a table.

Syntax:

```
drop table table_name ;
```

Example:

```
Sql> drop table emp ;
```

DATA MANIPULATION LANGUAGE (DML):

It is used to store, modify, retrieve, delete and update data in database.

a) SELECT Command:

The most commonly used sql command is select statement. Sql select statement is used to query or retrieve data from a table in the database. A query may retrieve information from specified

columns or from all of the columns in the table. To create a simple sql select Statement, you must specify the column(s) name and the table name. The whole query is called sql select Statement. A Basic syntax of select statement is:

Syntax:

```
select column_list from table-name ;
```

Example:

```
sql> select * from emp;
```

Select command is used with some clauses:

i. Where Clause:

The WHERE Clause is used when you want to retrieve specific information from a table excluding other irrelevant data.

Syntax:

```
select column_list from table-name  
where column_name = value;
```

Example:

```
sql> select * from emp      OR      sql > select * from emp  
      where eno = 101;          where ename = 'Ram' ;
```

ii. Order by clause:

The ORDER BY clause is used in a select statement to sort results either in ascending or descending order.

Syntax:

```
select column_list from table-name order by column_name asc / desc ;
```

Example:

```
sql > select * from emp order by ename desc ;
```

iii. Group by clause:

The sql group by Clause is used along with the group functions to retrieve data grouped according to one or more columns.

Syntax:

```
select column_list, function_name(column_name) from table-name  
group by column_name ;
```

Example:

If you want to know the total amount of salary spent on each department.

```
sql > select dept_name, sum(salary) from emp  
      group by dept_name ;
```

iv. Having clause:

Having clause is used to filter data based on the group functions. This is similar to where condition but is used with group functions. Group functions cannot be used in where Clause but can be used in having clause.

Syntax:

```
select column_list, function_name(column_name) from table-name
group by column_name
having condition ;
```

Example:

If you want to know the total amount of salary spent on each department.

```
sql > select edept_name, sum(salary) from emp
group by edept_name
having sum(salary) >= 10000;
```

b) INSERT Command:

The insert into statement is used to insert new records in a table. It is possible to write the insert into statement in three forms.

I) The first form specifies both the column names and the values to be inserted:

Syntax:

```
insert into table_name (column1,column2,column3,...) Values (value1,value2,value3,...) ;
```

Example:

```
sql> insert into emp(eno, ename, esal, ecity) values (101, 'amit das' , 15000, 'indore') ;
```

II) The second form does not specify the column names where the data will be inserted, only their values:

Syntax:

```
insert into table_name values (value1,value2,value3,...) ;
```

Example:

```
sql> insert into emp values (101, 'amit das' , 15000, 'indore') ;
```

III) The second form specifies both the column names and the address of column to be inserted.

Syntax:

```
insert into table_name (column1,column2,column3,...) Values
(&column1,&column2,column3,...) ;
```

Example:

```
Sql> insert into emp (eno, ename, esal, ecity) values (&eno , '& ename' , &esal , '&
ecity') ;
```

c) UPDATE Command:

The sql update Query is used to modify the existing records in a table.

I) If we want to update same value in one column. Then update command will be:

Syntax:

```
update table_name set column=value1, column=value1, .... ;
```

Example:

```
Sql> update emp set ecity = 'indore' ;
```

II) Update command with Where clause:

The where clause specifies which record or records that should be updated.

Syntax:

```
update table_name set column=value1, column=value1, ....
```


where column = value ;

Example:

```
Sql > update emp set ename= 'Ram Sharma' , esal = 15000 ;  
      where eno = 105 ;
```

d) DELETE Command:

I) The sql delete query is used to delete the existing records from a table. All the records would be deleted from table.

Syntax:

```
delete table table_name ;
```

Example:

```
Sql > delete table emp ;
```

II) Delete command with Where clause:

We can use where clause with delete query to delete selected rows.

Syntax:

```
delete table table_name  
where column = value ;
```

Example:

```
Sql > delete table emp  
      where eno = 105 ;
```

III) TRUNCATE Command:

We can use truncate command instead of delete. This command is similar to delete command. All the records would be deleted from table.

Syntax:

```
truncate table table_name ;
```

Example:

```
Sql > truncate table emp ;
```

DATA CONTROL LANGUAGE (DCL):

DCL commands are used to enforce database security in a multiple user database environment.

Only Database Administrator's or owners of the database object can provide/remove privileges on a database object.

Two types of DCL commands are

a) GRANT Command:

Grant is used to provide permissions like select, insert, Execute to user on the database objects like Tables,

Views, and Databases etc.

Syntax:

```
Grant privilegeName  
On objectName
```

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

To {userName/Public/roleName}

[with Grant Option]

Where :

- **privilege_name** is the access right or privilege granted to the user. Some of the access rights are all, execute, and select.

Privilege	Description
SELECT	Ability to perform SELECT statements on the table.
INSERT	Ability to perform INSERT statements on the table.
UPDATE	Ability to perform UPDATE statements on the table.
DELETE	Ability to perform DELETE statements on the table.
REFERENCES	Ability to create a constraint that refers to the table.
ALTER	Ability to perform ALTER TABLE statements to change the table definition.
INDEX	Ability to create an index on the table with the create index statement.
ALL	All privileges on table.

- **object_name** is the name of a database object like table, view, stored proc and sequence.
- **user_name** is the name of the user to whom an access right is being granted.
- **user_name** is the name of the user to whom an access right is being granted.
- **public** is used to grant access rights to all users.
- **roles** are a set of privileges grouped together.
- **with grant option** - allows a user to grant access rights to other users.

Example:

4. If we wanted to grant select, insert, update, and delete privileges on a table called suppliers to a user name kamal.

```
Sql> grant select, insert, update, delete
      on suppliers
      to kamal ;
```

5. We can also use the ALL keyword to indicate that you wish ALL permissions to be granted for a user named kamal.

```
Sql> grant ALL
      on suppliers
      to kamal ;
```

6. If we wanted to grant only SELECT access on your table to all users, you could grant the privileges to the public keyword.

```
Sql> grant select
      on suppliers
      to public ;
```

b) REVOKE Command:

The revoke command removes user access rights or privileges to the database objects.

Syntax:

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

```
revoke privilege_name
on object_name
from {user_name |public |role_name}
```

Example:

1. If you wanted to revoke DELETE privileges on a table called emp from a user named kamal.
Sql> revoke delete
on emp
from kamal ;
2. If you wanted to revoke ALL privileges on a table for a user named kamal.
Sql> revoke all
on emp
from kamal ;
3. If you had granted ALL privileges to public (all users) on the emp table and you wanted to revoke these privileges.
Sql> revoke all
on emp
from public ;

TRANSACTION CONTROL LANGUAGE (TCL):

It is used to control transactional processing in a database. A transaction is logical unit of work that comprises one or more SQL statements. There are following commands used to control transactions:

a) COMMIT Command:

A **COMMIT** ends the current transaction and makes permanent any changes made during the transaction. All

transaction locks acquired on tables are released.

Syntax:

```
Commit ;
```

Example:

```
Sql> update emp set esal = esal + 1500 ;
Sql > commit ;
```

b) ROLLBACK Command:

The rollback command is the transactional command used to undo transactions that have not already been

saved to the database. The rollback command can only be used to undo transactions since the last

commit or rollback command was issued.

Syntax:

```
rollback ;
```

Example:

```
Sql> delete from emp
where eno = 102 ;
Sql > rollback ;
```

c) SAVE POINT Command:

A SAVEPOINT is a point in a transaction when you can roll the transaction back to a certain point without rolling back the entire transaction.

Syntax:

Savepoint savepoint_name;

Example:

```
Sql> insert into emp values ( 108, 'lata', 12000, 'mhow');
Sql> insert into emp values ( 109, 'harish', 18500, 'jaipur');
Sql> savepoint xyz ;
Sql> update emp set esal = esal + 100 ;
Sql> rollback to savepoint_name ;
```

OTHER USEFULL COMMANDS:

1. DESC/ DESCRIBE command:

It provides information about the columns in a table.

Syntax:

desc table_name ;

Example:

```
Sql> desc emp;
```

Output:

name	null?	type
employee_id	not null	number(8)
ename		varchar(15)
esal		number(15)
ecity		varchar(12)

2. DISTINCT command:

In tables some of the columns may contain duplicate values. But you want to list out the (distinct) values of a

coloum in that table. The DISTINCT keyword can be used to return only distinct (different) values. There are

no duplicate value in column.

Syntax:

Select distinct (column_name) from table_name ;

Example:

```
Sql> select distinct (ename) from emp;
```

3. RENAME command:

If we want to change the table name then we use this command.

Syntax:

Rename table old_table_name to New_table_name ;

Example:

```
Sql> rename table emp to employee ;
```

4. ALIAS Command:

RENAISSANCE UNIVERSITY, INDORE

BCA III SEM

Subject:

Unit III: Database Management System

ALIASES can be used to create a temporary name for columns or tables.

- i. **Column alias name:** Aliases are used to make the column headings in your result set easier to read.

Syntax:

```
Select column_name(s) as alias_name from table_name ;
```

Example:

```
Sql> select empno as eno, empname as ename from emp ;
```

- ii. **Table alias name:** Aliases are used to make the table headings in your result set easier to read.

Syntax:

```
Select column_name from table_name alias_table_name ;
```

Example:

```
Sql> select e.ename, e.esal, d.dname from employee e, dept d  
where e.dno = d.dno;
```

5. RowNum Command:

If we want to access top 3 records from the table.

Syntax:

```
select * from table_name  
where rownum <= value;
```

Example:

```
Sql> select * from emp  
where rownum <= 3;
```

SET OPERATIONS:

Set operators combine results from two or more queries into a single result set.

There are three set operations:

7. UNOIN operator
8. INTERSECTION operator
9. MINUS operator

Table: Emp1		Table: Emp2	
Eno	Ename	Eno	Ename
101	Ram	103	Hari
102	Amit	104	Kamal
103	Hari	105	Rahul
104	Kamal	106	Dinesh

1. UNOIN operator:

Union is used to combine the result of two or more selected statements. In union, number of column and data type must be same in the both table. The UNION operator selects only distinct values by default.

Example:

```
Sql> select * from emp1  
union
```

```
select * from emp2;
```

Output

Eno	Ename
101	Ram
102	Amit
103	Hari
104	Kamal
105	Rahul
106	Dinesh

UNION ALL operator

Union is used to combine the result of two or more selected statements. In union, number of column and data type must be same in the both table. The UNION ALL operator selects all rows from both tables with duplicate values.

Example:

```
Sql> select * from emp1
      union all
      select * from emp2;
```

Output

Eno	Ename
101	Ram
102	Amit
103	Hari
104	Kamal
103	Hari
104	Kamal
105	Rahul
106	Dinesh

2. INTERSECTION operator:

INTERSECT returns only common rows returned by the two select statements. Each SQL statement within the SQL intersects must have the same number of fields in the result sets with similar data types.

Example:

```
Sql> select * from emp1
      intersect
      select * from emp2;
```

Output:

Eno	Ename
103	Hari
104	Kamal

3. MINUS operator:

The SQL minus operator is used to return all rows in the first select statement that are not returned in the second select statement. Each select statement within the minus query must have the same number of fields in the result sets with similar data types.

Example:

```
Sql> select * from emp1
      minus
      select * from emp2;
```

Output:

Eno	Ename
101	Ram
102	Amit

FUNCTION:

There are two type of function in sql.

SCALAR FUNCTIONS:

A scalar function operates on a single value and returns a single value.

1. Length():

The length functions return the length of char. Length calculates length using characters as defined by the input character set.

Syntax:

```
Select length (column_name) from table_name ;
```

Example:

```
Sql> select length (ename) from emp ;
```

2. upper():

It returns the string str with all characters changed to uppercase according to the current character set mapping.

Syntax:

```
Select upper (column_name) from table_name ;
```

Example:

```
Sql> select upper (ename) from emp ;
```

3. lower():

It returns the string str with all characters changed to lowercase according to the current character set mapping.

Syntax:

```
Select lower (column_name) from table_name ;
```

Example:

```
Sql> select lower (ename) from emp ;
```

4. concat():

It returns the string those results from concatenating the arguments.

Syntax:

```
Select upper (column_name, column_name) from table_name ;
```

Example:

```
Sql> select upper (efname, elname) from emp ;
```

5. initcap():

It returns char, with the first letter of each word in uppercase, all other letters in lowercase. Words are delimited by white space or characters that are not alphanumeric.

Syntax:

```
Select initcap (column_name) from table_name ;
```

Example:

```
Sql> select initcap (ename) from emp ;
```

6. substr():

The forms without a len argument return a substring from string str starting at position pos.

Syntax:

```
Select substr (column_name, pos) from table_name ;
```

Example:

```
Sql> select initcap (ename,3) from emp ;
```

AGGREGATE FUNCTIONS:

SQL has many built-in functions for performing processing on string or numeric data. SQL aggregate functions return a single value, calculated from values in a column. Aggregate functions are used to compute against a "returned column of numeric data" from your select statement. They basically summarize the results of a particular column of selected data.

Example:

Employee table is shown below:

Eno	Ename	Esal
101	Amit Sharma	15000
102	Rahul joshi	10000
103	Adarsh Jain	9000
104	Kamal Yadav	12000
105	Shubh Deep	8000

1. MIN(): It returns the smallest value in a given column.

Example:

```
Sql> select min(esal) from employee;
```

Output:

```
min (esal)
```

```
-----  
8000
```

2. MAX(): It returns the largest value in a given column.

Example:

```
Sql> select max(esal) from employee;
```

Output:

```
max (esal)
```

15000

3. **SUM()**: It returns the sum of the numeric values in a given column.

Example:

Sql> select sum(esal) from employee;

Output:

sum (esal)

54000

4. **AVG()**: It returns the average value of a given column.

Example:

Sql> select avg(esal) from employee;

Output:

avg (esal)

10800

5. **COUNT()**: It returns the total number of values in a given column.

Example:

Sql> select count(esal) from employee;

Output:

count (esal)

5

PROCEDURES:

A stored procedure or in simple a proc is a named PL/SQL block which performs one or more specific task. This is similar to a procedure in other programming languages.

A procedure has a header and a body. The header consists of the name of the procedure and the parameters or variables passed to the procedure. The body consists of declaration section, execution section and exception section similar to a general PL/SQL Block.

A procedure is similar to an anonymous PL/SQL Block but it is named for repeated usage.

The following example creates a simple procedure that displays the string 'Hello World!' on the screen when executed.

Example:

```
CREATE OR REPLACE PROCEDURE greetings
AS
BEGIN
    dbms_output.put_line('Hello World!');
END;
/
```

When above code is executed using SQL prompt, it will produce the following result:

Procedure created.

Reference Questions:

Q1. What is functional Dependency?

Q2. What is Normalization? Explain different Normal Forms?

Q3. What do you mean by database view and stored procedure?

Q4. Explain – group by, order by, between and distinct in SQL?

Q5. What is Join Operation? Explain different join operations?

Transaction:

Transaction in Database Management Systems (DBMS) can be defined as a set of logically related operations. It is the result of a request made by the user to access the contents of the database and perform operations on it. It consists of various operations and has various states in its completion journey. It also has some specific properties that must be followed to keep the database consistent.

Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

i) Read(X)

A read operation is used to read the value of X from the database and store it in a buffer in the main memory for further actions such as displaying that value. Such an operation is performed when a user wishes just to see any content of the database and not make any changes to it. For example, when a user wants to check his/her account's balance, a read operation would be performed on user's account balance from the database.

ii) Write(X)

A write operation is used to write the value to the database from the buffer in the main memory. For a write operation to be performed, first a read operation is performed to bring its value in buffer, and then some changes are made to it, e.g. some sets of arithmetic operations are performed on it according to the user's request, then to store the modified value back in the database, a write operation is performed. For example, when a user requests to withdraw some money from his account, his account balance is fetched from the database using a read operation, then the amount to be deducted from the account is subtracted from this value, and then the obtained value is stored back in the database using a write operation.

iii) Commit

This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction. To ensure that further operations of any other transaction are performed only after work of the current transaction is done, a commit operation is performed to the changes made by a transaction permanently to the database.

iv) Rollback

This operation is performed to bring the database to the last saved state when any transaction is interrupted in between due to any power, hardware, or software failure. In simple words, it can be said that a rollback operation does undo the operations of transactions that were performed before its interruption to achieve a safe state of the database and avoid any kind of ambiguity or inconsistency.

Properties of Transaction

As transactions deal with accessing and modifying the contents of the database, they must have some basic properties which help maintain the consistency and integrity of the database before and after the transaction. Transactions follow 4 properties, namely, Atomicity, Consistency, Isolation, and Durability. Generally, these are referred to as ACID properties of transactions in DBMS. ACID is the acronym used for transaction properties. A brief description of each property of the transaction is as follows.

i) Atomicity

This property ensures that either all operations of a transaction are executed or it is aborted. In any case, a transaction can never be completed partially. Each transaction is treated as a single unit (like an atom). Atomicity is achieved through commit and rollback operations, i.e. changes are made to the database only if all operations related to a transaction are completed, and if it gets interrupted, any changes made are rolled back using rollback operation to bring the database to its last saved state.

ii) Consistency

This property of a transaction keeps the database consistent before and after a transaction is completed. Execution of any transaction must ensure that after its execution, the database is either in its prior stable state or a new stable state. In other words, the result of a transaction should be the transformation of a database from one consistent state to another consistent state. Consistency, here means, that the changes made in the database are a result of logical operations only which the user desired to perform and there is not any ambiguity.

iii) Isolation

This property states that two transactions must not interfere with each other, i.e. if some data is used by a transaction for its execution, then any other transaction can not concurrently access that data until the first transaction has completed. It ensures that the integrity of the database is maintained and we don't get any ambiguous values. Thus, any two transactions are isolated from each other. This property is enforced by the concurrency control subsystem of DBMS.

iv) Durability

This property ensures that the changes made to the database after a transaction is completely executed, are durable. It indicates that permanent changes are made by the successful execution of a transaction. In the event of any system failures or crashes, the consistent state achieved after the completion of a transaction remains intact. The recovery subsystem of DBMS is responsible for enforcing this property.

Transaction States :

1. Active State –

When the instructions of the transaction are running then the transaction is in active state. If all the 'read and write' operations are performed without any error then it goes to the "partially committed state"; if any instruction fails, it goes to the "failed state".

2. Partially Committed –

After completion of all the read and write operation the changes are made in main memory or local buffer. If the changes are made permanent on the DataBase then the state will change to "committed state" and in case of failure it will go to the "failed state".

3. Failed State –

When any instruction of the transaction fails, it goes to the "failed state" or if failure occurs in making a permanent change of data on Data Base.

4. Aborted State –

After having any type of failure the transaction goes from "failed state" to "aborted state" and since in previous states, the changes are only made to local buffer or main memory and hence these changes are deleted or rolled-back.

5. Committed State –

It is the state when the changes are made permanent on the Data Base and the transaction is complete and therefore terminated in the "terminated state".

6. Terminated State –

If there isn't any roll-back or the transaction comes from the "committed state", then the system is consistent and ready for new transaction and the old transaction is terminated.

Concurrency Control:

Concurrency Control is the management procedure that is required for controlling concurrent execution of the operations that take place on a database.

1. In a multi-user system, multiple users can access and use the same database at one time, which is known as the concurrent execution of the database. It means that the same database is executed simultaneously on a multi-user system by different users.
2. While working on the database transactions, there occurs the requirement of using the database by multiple users for performing different operations, and in that case, concurrent execution of the database is performed.
3. The thing is that the simultaneous execution that is performed should be done in an interleaved manner, and no operation should affect the other executing operations, thus maintaining the consistency of the database. Thus, on making the concurrent execution of the transaction operations, there occur several challenging problems that need to be solved.

Problems with Concurrent Execution

In a database transaction, the two main operations are READ and WRITE operations. So, there is a need to manage these two operations in the concurrent execution of the transactions as if these operations are not performed in an interleaved manner, and the data may become inconsistent. So, the following problems occur with the Concurrent Execution of the operations:

Problem 1: Lost Update Problems (W - W Conflict)

The problem occurs when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.

Time	T _x	T _y
t ₁	READ (A)	—
t ₂	A = A - 50	—
t ₃	—	READ (A)
t ₄	—	A = A + 100
t ₅	—	—
t ₆	WRITE (A)	—
t ₇	—	WRITE (A)

LOST UPDATE PROBLEM

1. At time t1, transaction TX reads the value of account A, i.e., \$300 (only read).
2. At time t2, transaction TX deducts \$50 from account A that becomes \$250 (only deducted and not updated/write).
3. Alternately, at time t3, transaction TY reads the value of account A that will be \$300 only because TX didn't update the value yet.
4. At time t4, transaction TY adds \$100 to account A that becomes \$400 (only added but not updated/write).
5. At time t6, transaction TX writes the value of account A that will be updated as \$250 only, as TY didn't update the value yet.
6. Similarly, at time t7, transaction TY writes the values of account A, so it will write as done at time t4 that will be \$400. It means the value written by TX is lost, i.e., \$250 is lost.

Hence data becomes incorrect, and database sets to inconsistent.

Lock-Based Protocol

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:

1. Shared lock:

It is also known as a Read-only lock. In a shared lock, the data item can only read by the transaction.

It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item.

2. Exclusive lock:

In the exclusive lock, the data item can be both reads as well as written by the transaction.

This lock is exclusive, and in this lock, multiple transactions do not modify the same data simultaneously.

In a database management system (DBMS), a deadlock occurs when two or more transactions are waiting for each other to release resources, such as locks on database objects, that they need to complete their operations. As a result, none of the transactions can proceed, leading to a situation where they are stuck or “deadlocked.”

Deadlocks can happen in multi-user environments when two or more transactions are running concurrently and try to access the same data in a different order. When this happens, one transaction may hold a lock on a resource that another transaction needs, while the second transaction may hold a lock on a resource that the first transaction needs. Both transactions are then blocked, waiting for the other to release the resource they need.

DBMSs often use various techniques to detect and resolve deadlocks automatically. These techniques include timeout mechanisms, where a transaction is forced to release its locks after a certain period of time, and deadlock detection algorithms, which periodically scan the transaction log for deadlock cycles and then choose a transaction to abort to resolve the deadlock.

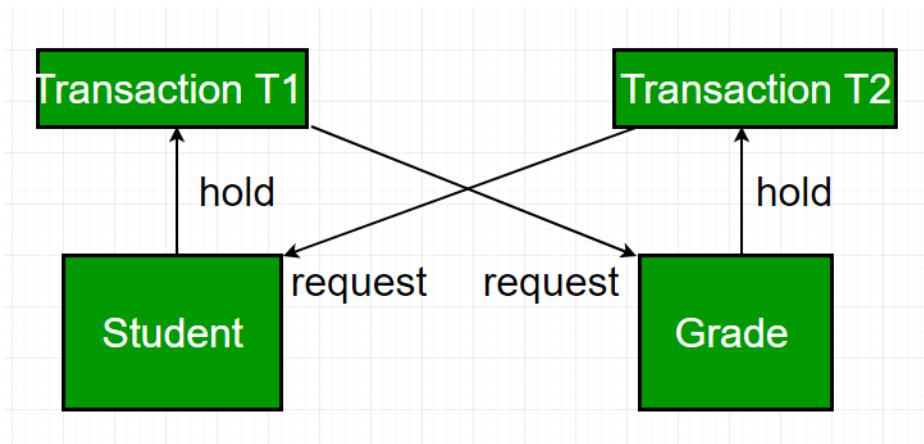
It is also possible to prevent deadlocks by careful design of transactions, such as always acquiring locks in the same order or releasing locks as soon as possible. Proper design of the database schema and application can also help to minimize the likelihood of deadlocks.

In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a Halt.

Example – let us understand the concept of Deadlock with an example :

Suppose, Transaction T1 holds a lock on some rows in the Students table and needs to update some rows in the Grades table. Simultaneously, Transaction T2 holds locks on those very rows (Which T1 needs to update) in the Grades table but needs to update the rows in the Student table held by Transaction T1.

Now, the main problem arises. Transaction T1 will wait for transaction T2 to give up the lock, and similarly, transaction T2 will wait for transaction T1 to give up the lock. As a consequence, All activity comes to a halt and remains at a standstill forever unless the DBMS detects the deadlock and aborts one of the transactions.



Deadlock prevention: For a large database, the deadlock prevention method is suitable. A deadlock can be prevented if the resources are allocated in such a way that a deadlock never occurs. The DBMS analyzes the operations whether they can create a deadlock situation or not, If they do, that transaction is never allowed to be executed.

Deadlock prevention mechanism proposes two schemes:

- **Wait-Die Scheme:** In this scheme, If a transaction requests a resource that is locked by another transaction, then the DBMS simply checks the timestamp of both transactions and allows the older transaction to wait until the resource is available for execution.

Suppose, there are two transactions T1 and T2, and Let the timestamp of any transaction T be TS (T). Now, If there is a lock on T2 by some other transaction and T1 is requesting resources held by T2, then DBMS performs the following actions:

Checks if $TS(T1) < TS(T2)$ – if T1 is the older transaction and T2 has held some resource, then it allows T1 to wait until resource is available for execution. That means if a younger transaction has locked some resource and an older transaction is waiting for it, then an older transaction is allowed to wait for it till it is available. If T1 is an older transaction and has held some resource with it and if T2 is waiting for it, then T2 is killed and restarted later with random delay but with the same timestamp. i.e. if the older transaction has held some resource and the younger transaction waits for the resource, then the younger transaction is killed and restarted with a very minute delay with the same timestamp.

This scheme allows the older transaction to wait but kills the younger one.

- **Wound Wait Scheme:** In this scheme, if an older transaction requests for a resource held by a younger transaction, then an older transaction forces a younger transaction to kill the transaction and release the resource. The younger transaction is restarted with a minute delay but with the same timestamp. If the younger transaction is requesting a

resource that is held by an older one, then the younger transaction is asked to wait till the older one releases it.

Reference Questions:

Q1. What is Transaction in DBMS?

Q2. What are ACID properties in DBMS?

Q3. What is Database Transaction? Explain different Transaction States?

Q4. Explain Concurrent Execution Problem? Also Explain its solutions?

Q5. What is DeadLock? How it can be Handled?

Database Security

Security of databases refers to the array of controls, tools, and procedures designed to ensure and safeguard confidentiality, integrity, and accessibility.

Security for databases must cover and safeguard the following aspects:

- The database containing data.
- Database management systems
- Any applications that are associated with it.
- Physical database servers or the database server virtual, and the hardware that runs it.
- The infrastructure for computing or network that is used to connect to the database.

Security of databases is a complicated and challenging task that requires all aspects of security practices and technologies. This is inherently at odds with the accessibility of databases. The more usable and accessible the database is, the more susceptible we are to threats from security. The more vulnerable it is to attacks and threats, the more difficult it is to access and utilize.

Why Database Security is Important?

According to the definition, a data breach refers to a breach of data integrity in databases. The amount of damage an incident like a data breach can cause our business is contingent on various consequences or elements.

- **Intellectual property that is compromised:** Our intellectual property--trade secrets, inventions, or proprietary methods -- could be vital for our ability to maintain an advantage in our industry. If our intellectual property has been stolen or disclosed and our competitive advantage is lost, it could be difficult to keep or recover.
- **The damage to our brand's reputation:** Customers or partners may not want to purchase goods or services from us (or deal with our business) If they do not feel they can trust our company to protect their data or their own.
- **The concept of business continuity (or lack of it):** Some businesses cannot continue to function until a breach has been resolved.
- **Penalties or fines to be paid for not complying:** The cost of not complying with international regulations like the Sarbanes-Oxley Act (SAO) or Payment Card Industry Data Security Standard (PCI DSS) specific to industry regulations on data privacy, like HIPAA or

regional privacy laws like the European Union's General Data Protection Regulation (GDPR) could be a major problem with fines in worst cases in excess of many million dollars for each violation.

- **Costs for repairing breaches and notifying consumers about them:** Alongside notifying customers of a breach, the company that has been breached is required to cover the investigation and forensic services such as crisis management, triage repairs to the affected systems, and much more.
-

Common Threats and Challenges

Numerous software configurations that are not correct, weaknesses, or patterns of carelessness or abuse can lead to a breach of security. Here are some of the most prevalent kinds of reasons for security attacks and the reasons.

Insider Dangers

An insider threat can be an attack on security from any three sources having an access privilege to the database.

- A malicious insider who wants to cause harm
- An insider who is negligent and makes mistakes that expose the database to attack. vulnerable to attacks
- An infiltrator is an outsider who acquires credentials by using a method like phishing or accessing the database of credential information in the database itself.

Insider dangers are among the most frequent sources of security breaches to databases. They often occur as a consequence of the inability of employees to have access to privileged user credentials.

Human Error

The unintentional mistakes, weak passwords or sharing passwords, and other negligent or uninformed behaviors of users remain the root causes of almost half (49 percent) of all data security breaches.

Database Software Vulnerabilities can be Exploited

Hackers earn their money by identifying and exploiting vulnerabilities in software such as databases management software. The major database software companies and open-source

databases management platforms release regular security patches to fix these weaknesses. However, failing to implement the patches on time could increase the risk of being hacked.

SQL/NoSQL Injection Attacks

A specific threat to databases is the infusing of untrue SQL as well as other non-SQL string attacks in queries for databases delivered by web-based apps and HTTP headers. Companies that do not follow the safe coding practices for web applications and conduct regular vulnerability tests are susceptible to attacks using these.

Buffer Overflow is a way to Exploit Buffers

Buffer overflow happens when a program seeks to copy more data into the memory block with a certain length than it can accommodate. The attackers may make use of the extra data, which is stored in adjacent memory addresses, to establish a basis for they can begin attacks.

DDoS (DoS/DDoS) Attacks

In a denial-of-service (DoS) attack in which the attacker overwhelms the targeted server -- in this case, the database server with such a large volume of requests that the server is unable to meet no longer legitimate requests made by actual users. In most cases, the server is unstable or even fails to function.

Malware

Malware is software designed to exploit vulnerabilities or cause harm to databases. Malware can be accessed via any device that connects to the databases network.

Attacks on Backups

Companies that do not protect backup data using the same rigorous controls employed to protect databases themselves are at risk of cyberattacks on backups.

The following factors amplify the threats:

- **Data volumes are growing:** Data capture, storage, and processing continue to increase exponentially in almost all organizations. Any tools or methods must be highly flexible to meet current as well as far-off needs.
- **The infrastructure is sprawling:** Network environments are becoming more complicated, especially as companies shift their workloads into multiple clouds and hybrid cloud

architectures and make the selection of deployment, management, and administration of security solutions more difficult.

- **More stringent requirements for regulatory compliance:** The worldwide regulatory compliance landscape continues to increase by complexity. This makes the compliance of every mandate more challenging.

Best use of Database Security:

As databases are almost always accessible via the network, any security risk to any component or part of the infrastructure can threaten the database. Likewise, any security attack that impacts a device or workstation could endanger the database. Therefore, security for databases must go beyond the limits of the database.

In evaluating the security of databases in our workplace to determine our organization's top priorities, look at each of these areas.

- **Security for physical security:** If the database servers are on-premises or the cloud data centre, they should be placed in a secure, controlled climate. (If our server for database is located in a cloud-based data centre, the cloud provider will handle the security on our behalf.)
- **Access to the network and administrative restrictions:** The practical minimum number of users granted access to the database and their access rights should be restricted to the minimum level required to fulfil their tasks. Additionally, access to the network is limited to the minimum permissions needed.
- **End security of the user account or device:** Be aware of who has access to the database and when and how data is used. Monitoring tools for data can notify you of data-related activities that are uncommon or seem to be dangerous. Any device that connects to the network hosting the database must be physically secured (in the sole control of the appropriate person) and be subject to security checks throughout the day.
- **Security:** ALL data--including data stored in databases, as well as credential information should be secured using the highest-quality encryption when in storage and while in transport. All encryption keys must be used in accordance with the best practices guidelines.

- **Security of databases using software:** Always use the most current version of our software to manage databases and apply any patches immediately after they're released.
- **Security for web server applications and websites:** Any application or web server that connects to the database could be a target and should be subjected to periodic security testing and best practices management.
- **Security of backups:** All backups, images, or copies of the database should have the identical (or equally rigorous) security procedures as the database itself.
- **Auditing:** Audits of security standards for databases should be conducted every few months. Record all the logins on the server as well as the operating system. Also, record any operations that are made on sensitive data, too.

Data protection tools and platforms:

Today, a variety of companies provide data protection platforms and tools. A comprehensive solution should have all of the following features:

- **Discovery:** The ability to discover is often needed to meet regulatory compliance requirements. Look for a tool that can detect and categorize weaknesses across our databases, whether they're hosted in the cloud or on-premises. It will also provide recommendations to address any vulnerabilities that are discovered.
- **Monitoring of Data Activity:** The solution should be capable of monitoring and analysing the entire data activity in all databases, whether our application is on-premises, in the cloud, or inside a container. It will alert us to suspicious activity in real-time to allow us to respond more quickly to threats. It also provides visibility into the state of our information through an integrated and comprehensive user interface. It is also important to choose a system that enforces rules that govern policies, procedures, and the separation of duties. Be sure that the solution we select is able to generate the reports we need to comply with the regulations.
- **The ability to Tokenize and Encrypt Data:** In case of an incident, encryption is an additional line of protection against any compromise. Any software we choose to use must have the flexibility to protect data cloud, on-premises hybrid, or multi-cloud environments. Find a tool with volume, file, and application encryption features that meet our company's

regulations for compliance. This could require tokenization (data concealing) or advanced key management of security keys.

- **Optimization of Data Security and Risk Analysis:** An application that will provide contextual insights through the combination of security data with advanced analytics will allow users to perform optimizing, risk assessment, and reporting in a breeze. Select a tool that is able to keep and combine large amounts of recent and historical data about the security and state of your databases. Also, choose a solution that provides data exploration, auditing, and reporting capabilities via an extensive but user-friendly self-service dashboard.

Storage System in DBMS:

A database system provides an ultimate view of the stored data. However, data in the form of bits, bytes get stored in different storage devices.

In this section, we will take an overview of various types of storage devices that are used for accessing and storing data.

Types of Data Storage

For storing the data, there are different types of storage options available. These storage types differ from one another as per the speed and accessibility. There are the following types of storage devices used for storing the data:

- Primary Storage
- Secondary Storage
- Tertiary Storage

Database users:

Database users are categorized based up on their interaction with the database. These are seven types of database users in DBMS.

1. **Database Administrator (DBA)** : Database Administrator (DBA) is a person/team who defines the schema and also controls the 3 levels of database. The DBA will then create a new account id and password for the user if he/she need to access the database. DBA is also responsible for providing security to the database and he allows only the authorized users to access/modify the data base. DBA is responsible for the problems such as security breaches and poor system response time.
 - DBA also monitors the recovery and backup and provide technical support.
 - The DBA has a DBA account in the DBMS which called a system or superuser account.
 - DBA repairs damage caused due to hardware and/or software failures.
 - DBA is the one having privileges to perform DCL (Data Control Language) operations such as GRANT and REVOKE, to allow/restrict a particular user from accessing the database.
2. **Naive / Parametric End Users** : Parametric End Users are the unsophisticated who don't have any DBMS knowledge but they frequently use the database applications in their daily life to get the desired results. For examples, Railway's ticket booking users are naive users. Clerks in any bank is a naive user because they don't have any DBMS knowledge but they still use the database and perform their given task.
3. **System Analyst** : System Analyst is a user who analyzes the requirements of parametric end users. They check whether all the requirements of end users are satisfied.
4. **Sophisticated Users** : Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database. They can develop their own database applications according to their requirement. They don't write the program code but they interact the database by writing SQL queries directly through the query processor.
5. **Database Designers** : Data Base Designers are the users who design the structure of database which includes tables, indexes, views, triggers, stored procedures and constraints which are usually enforced before the database is created or populated with data. He/she controls what data must be stored and how the data items to be related. It is responsibility of Database Designers to understand the requirements of different user groups and then create a design which satisfies the need of all the user groups.
6. **Application Programmers** : Application Programmers also referred as System Analysts or simply Software Engineers, are the back-end programmers who writes the code for the application programs. They are the computer professionals. These programs could be written in Programming languages such as Visual Basic, Developer, C, FORTRAN, COBOL etc. Application programmers design, debug, test, and maintain set of programs called "canned transactions" for the Naive (parametric) users in order to interact with database.

7. **Casual Users / Temporary Users** : Casual Users are the users who occasionally use/access the database but each time when they access the database they require the new information, for example, Middle or higher level manager.
8. **Specialized users** : Specialized users are sophisticated users who write specialized database application that does not fit into the traditional data-processing framework. Among these applications are computer aided-design systems, knowledge-base and expert systems etc.

Differed Data Modification and Immediate Data Modification.

1. **Deferred Update:** It is a technique for the maintenance of the transaction log files of the DBMS. It is also called NO-UNDO/REDO technique. It is used for the recovery of transaction failures that occur due to power, memory, or OS failures. Whenever any transaction is executed, the updates are not made immediately to the database. They are first recorded on the log file and then those changes are applied once the commit is done. This is called the “Re-doing” process. Once the rollback is done none of the changes are applied to the database and the changes in the log file are also discarded. If the commit is done before crashing the system, then after restarting the system the changes that have been recorded in the log file are thus applied to the database.
2. **Immediate Update:** It is a technique for the maintenance of the transaction log files of the DBMS. It is also called UNDO/REDO technique. It is used for the recovery of transaction failures that occur due to power, memory, or OS failures. Whenever any transaction is executed, the updates are made directly to the database and the log file is also maintained which contains both old and new values. Once the commit is done, all the changes get stored permanently in the database, and records in the log file are thus discarded. Once rollback is done the old values get restored in the database and all the changes made to the database are also discarded. This is called the “Un-doing” process. If the commit is done before crashing the system, then after restarting the system the changes are stored permanently in the database.

Reference Questions:

Q1. What is Database Failure? Explain different types of failures?

Q2. What is Database User? Explain the role of Database Administrator?

Q3. What do you mean by Atomicity?

Q4. Explain the reasons for Database Failure? How it can be Recovery.

RENAISSANCE UNIVERSITY

Department of Computer Science

BCA / BSc (cs) – Sem III

Database Management System Practical Assignment

Create the tables for following structures with appropriate datatype & size. And follow the given constraints.

Customers

Cust_ID	Name	Address	City	State	Zip

Constraints:

- a. **Cust_ID** must be Primary Key
- b. **Name** should not be NULL

Products

Prod_ID	Prod_Name	Price	Stock_Qty

Constraints:

- a. **Prod_ID** must be Primary Key
- b. **Stock_Qty** should not be Negative

Orders

Order_id	Cust_ID	Prod_ID	Sale_Rep	Order_Date	Qty

Constraints:

- a. **Order_ID** must be Primary Key
- b. **Cust_ID** should be set as Referential key with **Cust_ID** (**Customers**)
- c. **Prod_ID** should be set as Referential key with **Prod_ID** (**Products**)
- d. **Qty** should not be Negative and Zero

RENAISSANCE UNIVERSITY

Department of Computer Science

BCA / BSc (cs) – Sem III

Database Management System Practical Assignment

Using above tables perform following SQL operations:

1. Insert some relevant records in above tables.
2. Show the complete detail of each customer.
3. Show all product name with their price.
4. Retrieve name of all the sales representative working in organization.
5. Show the detail of all those customers who belongs to **Indore** in **Madhya Pradesh**.
6. Display the name, address of all those customers which belongs to either **Indore** or **Bhopal**.
7. Count total no. of customers which is not the belongs to **Indore** city.
8. Show Cust_ID, Name and Address of all the customers which is MP Domicile and belongs to either **Ujjain** or **Ratlam**
9. Count total no. of cities in each state.
10. Show detail of all the customers who live at '**Vijay Nagar**' in **Indore**
11. Display the Product with their price.
12. Show all those product name whose price is ranging between 3000 to 4500
13. Show the name of products whose stock is Null/Zero.
14. Retrieve the name of product whose stock is minimum.
15. Show the order details according to the order_date in ascending order.
16. Show the name of Sale Representative who has taken the order for order_id **101**
17. Show the product name for which 'AMIT' has taken the order as Sale Representative
18. Show the price of product for the order id **105**
19. Find out total no. of order made by the customer '**Vikram**'
20. Find out the total order is made on **25-Nov-24**